

CSE110A: Compilers

Topics:

- *Module 4 – Optimizing Code: CFGs, LVN, and SSA*
 - *Control Flow Graphs*
 - *SSA : Single Static Assignment (An optimization technique extensively used in LLVM)*
 - *SSA as LVN across Basic Blocks*

Optimizing Code: CFGs, LVN, and SSA

- Understanding the interplay in compiler optimizations

Introduction

- CFG (Control Flow Graph):
 - Representation of program paths
- LVN (Local Value Numbering):
 - Detects equivalent computations in basic blocks
 - SSA (Static Single Assignment): Each variable assigned exactly once, simplifies analysis

Control Flow Graphs (CFGs)

- - Nodes = Basic blocks, Edges = Control flow
- - Purpose: Identify paths, loops, optimize across blocks
- - Example textual diagram:

[B1] -> [B2] -> [B4]

\ /

-> [B3] --

Basic Blocks

- Sequence of instructions with single entry and exit
 - Characteristics:
 - No internal jumps
 - First instruction is a leader
- Role in optimizations: LVN operates within a block

Local Value Numbering (LVN)

- Goal: Detect redundant computations in a basic block
- Process:
 1. Assign a value number to each computation
 2. If an expression has been computed before, reuse the number
- Limitation: LVN is local, does not cross basic blocks
 - Example:
$$a = b + c \quad \# v1$$
$$x = b + c \quad \# \text{reuse } v1 \text{ as: } x = v1$$

Extending LVN Across Basic Blocks

- Challenge: Reuse computations across blocks

- Techniques:

- Use Available Expressions Analysis
- Integrate with CFG to propagate known values

- Example:

[B1]: $a = b + c$ (v1)

[B2]: $x = b + c$ reuse v1 if B1 dominates B2

Static Single Assignment (SSA)

- Each variable assigned once (unique Virtual Reg)
- Introduces ϕ -functions at merge points to unify multiple definitions
- Benefits:
- Makes LVN across blocks easier
 - Simplifies data-flow analyses
- - Example:
- [B2]: $a2 = \phi(a1, a0)$

SSA + LVN General Idea

- Combining SSA with LVN:
 - ϕ -functions clarify variable definitions
 - LVN can now operate globally without ambiguity

Example:

- [B1]: $t1 = b + c$ # First computation of $b + c$
- [B2]: $t3 = e + f$ # Another computation along a different path
- $t2 = \phi(t1, t3)$ # $t2$ takes $t1$ if path from B1, $t3$ if path from B2
- [B3]: $x = t2 + d$ # LVN can reuse $t1$ or $t3$ depending on path taken
- Notes:
 - ϕ -function merges values from different control paths
 - Each variable still assigned once, SSA guarantees clarity
 - LVN can detect redundancy even across blocks using SSA

Summary / Takeaways

- CFG provides structural framework
- LVN detects redundant computations locally
- SSA allows LVN to scale globally across blocks
- Together: powerful optimization pipeline

References / Further Reading

- Aho, Lam, Sethi, Ullman, Compilers: Principles, Techniques, and Tools
- Online tutorials on SSA, LVN, and CFG analysis